# SQL Practical No. 1 - Demonstrate GROUP BY and ORDER BY

```
QUESTION:

Demonstrate the use of GROUP BY and ORDER BY clause.

------------------------------------------------------------
STEP 1: CREATE SAMPLE TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_id NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Department VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om','IT',50000);
INSERT INTO EMPLOYEE VALUES (2,'Rahul','HR',30000);
INSERT INTO EMPLOYEE VALUES (3,'Sneha','IT',60000);
INSERT INTO EMPLOYEE VALUES (4,'Amit','Finance',40000);
INSERT INTO EMPLOYEE VALUES (5,'Neha','HR',35000);

COMMIT;

OUTPUT:
1 row inserted.
Commit complete.

------------------------------------------------------------
STEP 3: GROUP BY CLAUSE
------------------------------------------------------------

Find total salary department-wise:

SELECT Department, SUM(Salary) AS Total_Salary
FROM EMPLOYEE
GROUP BY Department;

EXPECTED OUTPUT:

Department | Total_Salary
IT         | 110000
HR         | 65000
Finance    | 40000

Explanation:
GROUP BY groups rows with same Department and calculates total salary.

------------------------------------------------------------
STEP 4: ORDER BY CLAUSE
------------------------------------------------------------

Display employees ordered by salary (descending):

SELECT * FROM EMPLOYEE
ORDER BY Salary DESC;

EXPECTED OUTPUT:

Emp_id | Emp_name | Department | Salary
3      | Sneha    | IT         | 60000
1      | Om       | IT         | 50000
4      | Amit     | Finance    | 40000
```

```
5       | Neha      | HR          | 35000
2       | Rahul     | HR          | 30000
```

Explanation:
ORDER BY sorts records in ascending (ASC) or descending (DESC) order.

---

VIVA POINTS:

1. GROUP BY is used with aggregate functions (SUM, COUNT, AVG, MAX, MIN).
2. ORDER BY sorts the result in ascending (default) or descending order.
3. GROUP BY is executed before ORDER BY in query processing.

# SQL Practical No. 2 - Hospital Database

```
QUESTION:

Consider the following schema for a Hospital Database:

DOCTOR (Did, Dname, DAddress, Qualification)
PATIENTMASTER (Pcode, Pname, Padd, age, gender, bloodgroup, Pid)
ADMITTEDPATIENT (Pcode, EntryDate, DischargeDate, WardNo, Disease)

Queries:
a) Find the detail of the doctor who is treating the patient of ward no 3
b) Find the detail of patient admitted within period 03/03/2020 to 25/05/2020
c) Find the detail of patient who are suffered from blood cancer
d) Create view on DOCTOR & PATIENTMASTER tables

------------------------------------------------------------
STEP 1: CREATE TABLES
------------------------------------------------------------

CREATE TABLE DOCTOR (
    Did NUMBER PRIMARY KEY,
    Dname VARCHAR2(50),
    DAddress VARCHAR2(100),
    Qualification VARCHAR2(30)
);

CREATE TABLE PATIENTMASTER (
    Pcode NUMBER PRIMARY KEY,
    Pname VARCHAR2(50),
    Padd VARCHAR2(100),
    age NUMBER,
    gender CHAR(1),
    bloodgroup VARCHAR2(5),
    Pid NUMBER,
    FOREIGN KEY (Pid) REFERENCES DOCTOR(Did)
);

CREATE TABLE ADMITTEDPATIENT (
    Pcode NUMBER,
    EntryDate DATE,
    DischargeDate DATE,
    WardNo NUMBER,
    Disease VARCHAR2(50),
    FOREIGN KEY (Pcode) REFERENCES PATIENTMASTER(Pcode)
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO DOCTOR VALUES (1,'Dr. Amit','Mumbai','MBBS');
INSERT INTO DOCTOR VALUES (2,'Dr. Neha','Pune','MD');

INSERT INTO PATIENTMASTER VALUES (101,'Rahul','Mumbai',25,'M','A+',1);
INSERT INTO PATIENTMASTER VALUES (102,'Sneha','Pune',40,'F','B+',2);

INSERT INTO ADMITTEDPATIENT VALUES
(101, TO_DATE('05-03-2020','DD-MM-YYYY'),
 TO_DATE('15-03-2020','DD-MM-YYYY'), 3, 'Fever');

INSERT INTO ADMITTEDPATIENT VALUES
(102, TO_DATE('10-04-2020','DD-MM-YYYY'),
 TO_DATE('20-05-2020','DD-MM-YYYY'), 2, 'Blood Cancer');

COMMIT;

------------------------------------------------------------
STEP 3: QUERIES
------------------------------------------------------------
a) Doctor treating patient in ward no 3
```

```
SELECT D.*
FROM DOCTOR D
JOIN PATIENTMASTER P ON D.Did = P.Pid
JOIN ADMITTEDPATIENT A ON P.Pcode = A.Pcode
WHERE A.WardNo = 3;

b) Patients admitted between 03/03/2020 and 25/05/2020

SELECT *
FROM ADMITTEDPATIENT
WHERE EntryDate BETWEEN
TO_DATE('03-03-2020','DD-MM-YYYY')
AND TO_DATE('25-05-2020','DD-MM-YYYY');

c) Patients suffering from blood cancer

SELECT *
FROM ADMITTEDPATIENT
WHERE Disease = 'Blood Cancer';

d) Create View on DOCTOR & PATIENTMASTER

CREATE VIEW Doctor_Patient_View AS
SELECT D.Dname, D.Qualification, P.Pname, P.age, P.bloodgroup
FROM DOCTOR D
JOIN PATIENTMASTER P ON D.Did = P.Pid;

------------------------------------------------------------
EXPECTED OUTPUT (Sample)

Doctor in Ward 3:
Did | Dname    | DAddress | Qualification
1   | Dr. Amit | Mumbai   | MBBS

Patients between given dates:
101 | 05-MAR-20 | 15-MAR-20 | 3 | Fever
102 | 10-APR-20 | 20-MAY-20 | 2 | Blood Cancer

View Created Successfully.

------------------------------------------------------------
VIVA POINTS:

1. PRIMARY KEY uniquely identifies records.
2. FOREIGN KEY creates relationship between tables.
3. JOIN is used to combine data from multiple tables.
4. VIEW is virtual table created from SELECT query.
```

# Practical No. 3

## Create Department table with the following structure and perform the operations:

***Question: Create department table with required structure.***

***Query:***

```
CREATE TABLE Department (
Deptno INT PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL,
Location VARCHAR(30),
Contact VARCHAR(15)
);
```

Output: Table created successfully.

***a) Add column Designation to the department table.***

***Query:***

```
ALTER TABLE Department
ADD Designation VARCHAR(30);
```

Output: Table altered successfully.

***b) Insert values into the table.***

***Query:***

```
INSERT INTO Department (Deptno, DeptName, Location, Contact, Designation)
VALUES
(1, 'Computer', 'Jalgaon', '9876543210', 'Head Dept'),
(2, 'Mechanical', 'Pune', '9876501234', 'Supervisor'),
(9, 'Civil', 'Mumbai', '9123456780', 'Manager');
```

***Output:***

| Deptno | DeptName | Location | Contact | Designation |
|---|---|---|---|---|
| 1 | Computer | Jalgaon | 9876543210 | Head Dept |
| 2 | Mechanical | Pune | 9876501234 | Supervisor |
| 9 | Civil | Mumbai | 9123456780 | Manager |

***c) List the records of dept table grouped by deptno.***

***Query:***

```
SELECT Deptno, COUNT(*) AS Total
FROM Department
GROUP BY Deptno;
```

***Output:***

| Deptno | Total |
|---|---|
| 1 | 1 |
| 2 | 1 |
| 9 | 1 |

***d) Update the record where deptno is 9.***

***Query:***

```
UPDATE Department
SET Location = 'Nashik',
Contact = '9999999999'
WHERE Deptno = 9;
```

***Output:***

| Deptno | DeptName | Location | Contact | Designation |
|---|---|---|---|---|
| 1 | Computer | Jalgaon | 9876543210 | Head Dept |
| 2 | Mechanical | Pune | 9876501234 | Supervisor |
| 9 | Civil | Nashik | 9999999999 | Manager |

***e) Delete any column data from the table.***

***Query:***

```
UPDATE Department
SET Contact = NULL
WHERE Deptno = 2;
```

***Output:***

| Deptno | DeptName | Location | Contact | Designation |
|---|---|---|---|---|
| 1 | Computer | Jalgaon | 9876543210 | Head Dept |
| 2 | Mechanical | Pune | NULL | Supervisor |
| 9 | Civil | Nashik | 9999999999 | Manager |

# SQL Practical 4 - Doctor & Patient Database

```
QUESTION:

Create database using following schema. Apply Integrity Constraints and
answer the following queries using SQL.

DOCTOR (Did, Dname, Daddress, qualification)
PATIENT (Pid, Pname, age, gender)

Integrity Constraints:
1. All attributes should NOT NULL.
2. Did should be UNIQUE.
3. Pid should be UNIQUE.
4. Gender should be 'M' or 'F'.

------------------------------------------------------------
STEP 1: CREATE TABLES
------------------------------------------------------------

CREATE TABLE DOCTOR (
    Did NUMBER PRIMARY KEY,
    Dname VARCHAR2(50) NOT NULL,
    Daddress VARCHAR2(100) NOT NULL,
    qualification VARCHAR2(20) NOT NULL
);

CREATE TABLE PATIENT (
    Pid NUMBER PRIMARY KEY,
    Pname VARCHAR2(50) NOT NULL,
    age NUMBER NOT NULL,
    gender CHAR(1) CHECK (gender IN ('M','F')) NOT NULL
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT AT LEAST 10 RECORDS
------------------------------------------------------------

INSERT INTO DOCTOR VALUES (1,'Dr. Amit','Mumbai','MBBS');
INSERT INTO DOCTOR VALUES (2,'Dr. Neha','Pune','MD');
INSERT INTO DOCTOR VALUES (3,'Dr. Raj','Delhi','M.S');
INSERT INTO DOCTOR VALUES (4,'Dr. Simran','Nashik','MBBS');
INSERT INTO DOCTOR VALUES (5,'Dr. Karan','Nagpur','M.D');

INSERT INTO PATIENT VALUES (1,'Rahul',25,'M');
INSERT INTO PATIENT VALUES (2,'Sneha',30,'F');
INSERT INTO PATIENT VALUES (3,'Aman',45,'M');
INSERT INTO PATIENT VALUES (4,'Pooja',35,'F');
INSERT INTO PATIENT VALUES (5,'Rohit',50,'M');

COMMIT;

OUTPUT:
1 row inserted.
Commit complete.

------------------------------------------------------------
STEP 3: QUERIES
------------------------------------------------------------

a) Insert at least 10 records
   (Done above)

b) Find details of all doctors
   SELECT * FROM DOCTOR;

c) Delete records from DOCTOR where qualification is 'M.S'
   DELETE FROM DOCTOR WHERE qualification='M.S';

d) Find details of patient where age < 40
   SELECT * FROM PATIENT WHERE age < 40;
```

e) Update patient name where patient id = 5
   UPDATE PATIENT SET Pname='Ramesh' WHERE Pid=5;

COMMIT;

---

EXPECTED OUTPUT (Sample)

SELECT * FROM DOCTOR;

| Did | Dname | Daddress | qualification |
|---|---|---|---|
| 1 | Dr. Amit | Mumbai | MBBS |
| 2 | Dr. Neha | Pune | MD |
| 4 | Dr. Simran | Nashik | MBBS |
| 5 | Dr. Karan | Nagpur | M.D |

SELECT * FROM PATIENT WHERE age < 40;

| Pid | Pname | age | gender |
|---|---|---|---|
| 1 | Rahul | 25 | M |
| 2 | Sneha | 30 | F |
| 4 | Pooja | 35 | F |

After Update:
SELECT * FROM PATIENT WHERE Pid=5;

5 | Ramesh | 50 | M

---

VIVA POINTS:

1. PRIMARY KEY ensures uniqueness and NOT NULL.
2. CHECK constraint restricts gender values.
3. NOT NULL prevents empty values.
4. DELETE removes records permanently.
5. UPDATE modifies existing records.

# PL/SQL Practical Examination

```
Question:
Write a PL/SQL code to create an employee database with the tables
and fields specified as below.

Employee [Emp_no, Employee_name, Street, City]

Works [Emp_no, Company_name, Joining_date, Designation, Salary]

Company [Emp_no, City]

Manages [Emp_no, Manager_name, Mang_no]


-----------------------------------------------
STEP 1: CREATE TABLES
-----------------------------------------------

CREATE TABLE Employee (
    Emp_no NUMBER PRIMARY KEY,
    Employee_name VARCHAR2(50),
    Street VARCHAR2(50),
    City VARCHAR2(50)
);

CREATE TABLE Works (
    Emp_no NUMBER,
    Company_name VARCHAR2(50),
    Joining_date DATE,
    Designation VARCHAR2(50),
    Salary NUMBER(8,2),
    PRIMARY KEY (Emp_no),
    FOREIGN KEY (Emp_no) REFERENCES Employee(Emp_no)
);

CREATE TABLE Company (
    Emp_no NUMBER,
    City VARCHAR2(50),
    PRIMARY KEY (Emp_no),
    FOREIGN KEY (Emp_no) REFERENCES Employee(Emp_no)
);

CREATE TABLE Manages (
    Emp_no NUMBER,
    Manager_name VARCHAR2(50),
    Mang_no NUMBER,
    PRIMARY KEY (Emp_no),
    FOREIGN KEY (Emp_no) REFERENCES Employee(Emp_no)
);

OUTPUT:
Table Created.

-----------------------------------------------
STEP 2: INSERT SAMPLE DATA
-----------------------------------------------

INSERT INTO Employee VALUES (1,'Om','MG Road','Mumbai');

INSERT INTO Works VALUES (1,'TCS',
TO_DATE('01-01-2024','DD-MM-YYYY'),
'Developer',40000);

INSERT INTO Company VALUES (1,'Mumbai');

INSERT INTO Manages VALUES (1,'Amit',101);

COMMIT;

OUTPUT:
1 row inserted.
Commit complete.
```

```
-----------------------------------------------
STEP 3: DISPLAY DATA
-----------------------------------------------

SELECT * FROM Employee;
SELECT * FROM Works;
SELECT * FROM Company;
SELECT * FROM Manages;

EXPECTED OUTPUT:

EMPLOYEE
Emp_no | Employee_name | Street   | City
1      | Om            | MG Road  | Mumbai

WORKS
Emp_no | Company_name | Joining_date | Designation | Salary
1      | TCS          | 01-JAN-24    | Developer   | 40000

COMPANY
Emp_no | City
1      | Mumbai

MANAGES
Emp_no | Manager_name | Mang_no
1      | Amit         | 101

-----------------------------------------------
VIVA POINTS
-----------------------------------------------

1. Emp_no is Primary Key in Employee table.
2. Foreign Key is used to link related tables.
3. Joining_date datatype is DATE.
4. Salary datatype is NUMBER(8,2).
```

# PL/SQL Practical No. 6 - Retrieve Employee Details

```
QUESTION:

Write a PL/SQL code to retrieve the employee name, joining date,
and designation from employee database of an employee whose
employee number is input by the user.

------------------------------------------------------------
STEP 1: CREATE TABLE (If not already created)
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_no NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Join_date DATE,
    Designation VARCHAR2(50)
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om',TO_DATE('01-01-2023','DD-MM-YYYY'),'Developer');
INSERT INTO EMPLOYEE VALUES (2,'Rahul',TO_DATE('15-06-2022','DD-MM-YYYY'),'Manager');
INSERT INTO EMPLOYEE VALUES (3,'Sneha',TO_DATE('10-03-2021','DD-MM-YYYY'),'HR');

COMMIT;

------------------------------------------------------------
STEP 3: PL/SQL BLOCK
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    v_empno EMPLOYEE.Emp_no%TYPE;
    v_name EMPLOYEE.Emp_name%TYPE;
    v_date EMPLOYEE.Join_date%TYPE;
    v_desig EMPLOYEE.Designation%TYPE;
BEGIN
    v_empno := &Enter_Employee_Number;

    SELECT Emp_name, Join_date, Designation
    INTO v_name, v_date, v_desig
    FROM EMPLOYEE
    WHERE Emp_no = v_empno;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Joining Date: ' || v_date);
    DBMS_OUTPUT.PUT_LINE('Designation: ' || v_desig);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
END;
/

------------------------------------------------------------
SAMPLE INPUT:

Enter value for Enter_Employee_Number: 1

------------------------------------------------------------
EXPECTED OUTPUT:

Employee Name: Om
Joining Date: 01-JAN-23
Designation: Developer

------------------------------------------------------------
```

VIVA POINTS:

1. %TYPE is used to inherit datatype from table column.
2. SELECT INTO is used to fetch single row data.
3. DBMS_OUTPUT.PUT_LINE prints output.
4. EXCEPTION handles runtime errors like NO_DATA_FOUND.

# PL/SQL Practical No. 7 - Update Salary Using Cursor

```
QUESTION:

Write a PL/SQL code to update the salary of employees who earn
less than the average salary using cursor.

------------------------------------------------------------
STEP 1: CREATE TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_id NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om',30000);
INSERT INTO EMPLOYEE VALUES (2,'Rahul',50000);
INSERT INTO EMPLOYEE VALUES (3,'Sneha',20000);
INSERT INTO EMPLOYEE VALUES (4,'Amit',60000);
INSERT INTO EMPLOYEE VALUES (5,'Neha',25000);

COMMIT;

------------------------------------------------------------
STEP 3: PL/SQL CODE USING CURSOR
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    v_avg_salary EMPLOYEE.Salary%TYPE;

    CURSOR emp_cursor IS
        SELECT Emp_id, Salary
        FROM EMPLOYEE
        WHERE Salary < (SELECT AVG(Salary) FROM EMPLOYEE);

BEGIN
    -- Calculate average salary
    SELECT AVG(Salary) INTO v_avg_salary FROM EMPLOYEE;

    DBMS_OUTPUT.PUT_LINE('Average Salary: ' || v_avg_salary);

    -- Loop through employees earning less than average
    FOR emp_rec IN emp_cursor LOOP
        UPDATE EMPLOYEE
        SET Salary = Salary + 5000
        WHERE Emp_id = emp_rec.Emp_id;

        DBMS_OUTPUT.PUT_LINE('Salary Updated for Emp_id: ' || emp_rec.Emp_id);
    END LOOP;

    COMMIT;
END;
/

------------------------------------------------------------
EXPECTED OUTPUT (Sample)

Average Salary: 37000

Salary Updated for Emp_id: 1
```

```
Salary Updated for Emp_id: 3
Salary Updated for Emp_id: 5

------------------------------------------------------------
VERIFY UPDATED DATA

SELECT * FROM EMPLOYEE;

Emp_id | Emp_name | Salary
1      | Om       | 35000
2      | Rahul    | 50000
3      | Sneha    | 25000
4      | Amit     | 60000
5      | Neha     | 30000

------------------------------------------------------------
VIVA POINTS:

1. Cursor is used to process multiple rows one by one.
2. AVG() is an aggregate function.
3. FOR LOOP automatically opens, fetches, and closes the cursor.
4. COMMIT saves the changes permanently.
```

# PL/SQL Practical No. 8 - Row Trigger on Salary Table

```
QUESTION:

Write a row trigger to insert the existing values of the salary table
into a new table when the salary table is updated.

------------------------------------------------------------
STEP 1: CREATE SALARY TABLE
------------------------------------------------------------

CREATE TABLE SALARY (
    Emp_id NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO SALARY VALUES (1,'Om',30000);
INSERT INTO SALARY VALUES (2,'Rahul',50000);
INSERT INTO SALARY VALUES (3,'Sneha',40000);

COMMIT;

------------------------------------------------------------
STEP 3: CREATE BACKUP TABLE
------------------------------------------------------------

CREATE TABLE SALARY_BACKUP (
    Emp_id NUMBER,
    Emp_name VARCHAR2(50),
    Old_Salary NUMBER,
    Updated_On DATE
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 4: CREATE ROW LEVEL TRIGGER
------------------------------------------------------------

CREATE OR REPLACE TRIGGER Salary_Update_Trigger
BEFORE UPDATE ON SALARY
FOR EACH ROW
BEGIN
    INSERT INTO SALARY_BACKUP
    VALUES (:OLD.Emp_id, :OLD.Emp_name, :OLD.Salary, SYSDATE);
END;
/

OUTPUT:
Trigger Created.

------------------------------------------------------------
STEP 5: UPDATE SALARY TABLE
------------------------------------------------------------

UPDATE SALARY
SET Salary = 35000
WHERE Emp_id = 1;

COMMIT;

------------------------------------------------------------
```

VERIFY BACKUP TABLE

---

```
SELECT * FROM SALARY_BACKUP;
```

EXPECTED OUTPUT:

```
Emp_id | Emp_name | Old_Salary | Updated_On
1      | Om       | 30000      | 22-FEB-26
```

---

VIVA POINTS:

1. Row Trigger executes once for each row affected.
2. :OLD keyword stores old values before update.
3. :NEW keyword stores new values after update.
4. BEFORE UPDATE trigger runs before data modification.
5. SYSDATE returns current system date.

# PL/SQL Practical No. 9 - Trigger to Show Old and New Ename

```
QUESTION:

Write a trigger on the Employee table which shows the old values
and new values of Ename after any updation on Ename on Employee table.

------------------------------------------------------------
STEP 1: CREATE EMPLOYEE TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_id NUMBER PRIMARY KEY,
    Ename VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om',30000);
INSERT INTO EMPLOYEE VALUES (2,'Rahul',40000);

COMMIT;

------------------------------------------------------------
STEP 3: CREATE ROW LEVEL TRIGGER
------------------------------------------------------------

SET SERVEROUTPUT ON;

CREATE OR REPLACE TRIGGER Emp_Name_Update_Trigger
BEFORE UPDATE OF Ename ON EMPLOYEE
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Old Name: ' || :OLD.Ename);
    DBMS_OUTPUT.PUT_LINE('New Name: ' || :NEW.Ename);
END;
/

OUTPUT:
Trigger Created.

------------------------------------------------------------
STEP 4: UPDATE EMPLOYEE NAME
------------------------------------------------------------

UPDATE EMPLOYEE
SET Ename = 'Bhavesh'
WHERE Emp_id = 1;

COMMIT;

------------------------------------------------------------
EXPECTED OUTPUT:

Old Name: Om
New Name: Bhavesh

------------------------------------------------------------
VIVA POINTS:

1. BEFORE UPDATE OF Ename trigger fires only when Ename is updated.
2. :OLD stores old column value.
3. :NEW stores new column value.
4. FOR EACH ROW makes it a row-level trigger.
5. DBMS_OUTPUT.PUT_LINE displays the result.
```

# PL/SQL Practical No. 10 - Procedure for Percentage Range

```
QUESTION:

Write a PL/SQL procedure to find the number of students ranging from
100-70%, 69-60%, 59-50% & below 49% in each course from the
STUDENT_COURSE table given by the procedure as parameter.

------------------------------------------------------------
STEP 1: CREATE TABLE
------------------------------------------------------------

CREATE TABLE STUDENT_COURSE (
    Student_id NUMBER PRIMARY KEY,
    Student_name VARCHAR2(50),
    Course VARCHAR2(50),
    Percentage NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO STUDENT_COURSE VALUES (1,'Om','MCA',85);
INSERT INTO STUDENT_COURSE VALUES (2,'Rahul','MCA',72);
INSERT INTO STUDENT_COURSE VALUES (3,'Sneha','MCA',65);
INSERT INTO STUDENT_COURSE VALUES (4,'Amit','MCA',55);
INSERT INTO STUDENT_COURSE VALUES (5,'Neha','MCA',45);
INSERT INTO STUDENT_COURSE VALUES (6,'Karan','BCA',78);
INSERT INTO STUDENT_COURSE VALUES (7,'Pooja','BCA',62);
INSERT INTO STUDENT_COURSE VALUES (8,'Riya','BCA',48);

COMMIT;

------------------------------------------------------------
STEP 3: CREATE PROCEDURE
------------------------------------------------------------

SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Student_Percentage_Report(p_course IN VARCHAR2)
IS
    v_70_100 NUMBER;
    v_60_69  NUMBER;
    v_50_59  NUMBER;
    v_below49 NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_70_100
    FROM STUDENT_COURSE
    WHERE Course = p_course AND Percentage BETWEEN 70 AND 100;

    SELECT COUNT(*) INTO v_60_69
    FROM STUDENT_COURSE
    WHERE Course = p_course AND Percentage BETWEEN 60 AND 69;

    SELECT COUNT(*) INTO v_50_59
    FROM STUDENT_COURSE
    WHERE Course = p_course AND Percentage BETWEEN 50 AND 59;

    SELECT COUNT(*) INTO v_below49
    FROM STUDENT_COURSE
    WHERE Course = p_course AND Percentage < 49;

    DBMS_OUTPUT.PUT_LINE('Course: ' || p_course);
    DBMS_OUTPUT.PUT_LINE('70-100% Students: ' || v_70_100);
    DBMS_OUTPUT.PUT_LINE('60-69% Students: ' || v_60_69);
    DBMS_OUTPUT.PUT_LINE('50-59% Students: ' || v_50_59);
    DBMS_OUTPUT.PUT_LINE('Below 49% Students: ' || v_below49);
```

```
END;
/

OUTPUT:
Procedure Created.

------------------------------------------------------------
STEP 4: EXECUTE PROCEDURE
------------------------------------------------------------

EXEC Student_Percentage_Report('MCA');

------------------------------------------------------------
EXPECTED OUTPUT:

Course: MCA
70-100% Students: 2
60-69% Students: 1
50-59% Students: 1
Below 49% Students: 1

------------------------------------------------------------
VIVA POINTS:

1. Procedure is a named PL/SQL block.
2. IN parameter is used to pass value to procedure.
3. COUNT(*) counts number of records.
4. BETWEEN operator checks range of values.
5. DBMS_OUTPUT.PUT_LINE displays output.
```

# PL/SQL Practical No. 11 - Stored Function for Addition

```
QUESTION:

Create a stored function that accepts 2 numbers and returns the
addition of passed values. Also, write the code to call your function.

------------------------------------------------------------
STEP 1: CREATE FUNCTION
------------------------------------------------------------

CREATE OR REPLACE FUNCTION Add_Numbers(
    num1 IN NUMBER,
    num2 IN NUMBER
) RETURN NUMBER
IS
    result NUMBER;
BEGIN
    result := num1 + num2;
    RETURN result;
END;
/

OUTPUT:
Function Created.

------------------------------------------------------------
STEP 2: CALL FUNCTION USING PL/SQL BLOCK
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    v_result := Add_Numbers(10, 20);
    DBMS_OUTPUT.PUT_LINE('Addition Result: ' || v_result);
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:

Addition Result: 30

------------------------------------------------------------
STEP 3: CALL FUNCTION USING SELECT STATEMENT
------------------------------------------------------------

SELECT Add_Numbers(15, 25) AS Result FROM DUAL;

------------------------------------------------------------
EXPECTED OUTPUT:

Result
------
40

------------------------------------------------------------
VIVA POINTS:

1. Function must return a value using RETURN keyword.
2. IN parameter is used to pass values to function.
3. DUAL is a dummy table used in Oracle.
4. Stored function can be called inside SELECT statement.
5. Difference between Procedure and Function:
   - Function returns value.
   - Procedure may or may not return value.
```

# PL/SQL Practical No. 12 - Function for Department Total Salary

```
QUESTION:

Write a PL/SQL function that accepts the department number and
returns the total salary of the department. Also, write a block to call the function.

------------------------------------------------------------
STEP 1: CREATE TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_id NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Dept_no NUMBER,
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om',10,30000);
INSERT INTO EMPLOYEE VALUES (2,'Rahul',10,40000);
INSERT INTO EMPLOYEE VALUES (3,'Sneha',20,50000);
INSERT INTO EMPLOYEE VALUES (4,'Amit',20,35000);

COMMIT;

------------------------------------------------------------
STEP 3: CREATE FUNCTION
------------------------------------------------------------

CREATE OR REPLACE FUNCTION Get_Total_Salary(p_deptno IN NUMBER)
RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT SUM(Salary) INTO v_total
    FROM EMPLOYEE
    WHERE Dept_no = p_deptno;

    RETURN v_total;
END;
/

OUTPUT:
Function Created.

------------------------------------------------------------
STEP 4: CALL FUNCTION USING PL/SQL BLOCK
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    v_result NUMBER;
BEGIN
    v_result := Get_Total_Salary(10);
    DBMS_OUTPUT.PUT_LINE('Total Salary of Department 10: ' || v_result);
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:

Total Salary of Department 10: 70000
------------------------------------------------------------
```

STEP 5: CALL FUNCTION USING SELECT STATEMENT

```
SELECT Get_Total_Salary(20) AS Total_Salary FROM DUAL;
```

EXPECTED OUTPUT:

```
Total_Salary
------------
85000
```

VIVA POINTS:

1. Function returns a value using RETURN statement.
2. SUM() is an aggregate function.
3. Parameter is passed using IN mode.
4. DUAL is a dummy table in Oracle.
5. Function can be called inside SELECT statement.

# PL/SQL Practical No. 13 - Package Specification and Body

```
QUESTION:

Write a PL/SQL code to create:

1. Package Specification
2. Package Body

For the insert, retrieve, update, and delete operations on a STUDENT table.

------------------------------------------------------------
STEP 1: CREATE STUDENT TABLE
------------------------------------------------------------

CREATE TABLE STUDENT (
    Student_id NUMBER PRIMARY KEY,
    Student_name VARCHAR2(50),
    Course VARCHAR2(50),
    Marks NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: CREATE PACKAGE SPECIFICATION
------------------------------------------------------------

CREATE OR REPLACE PACKAGE Student_Pkg AS

    PROCEDURE Insert_Student(
        p_id NUMBER,
        p_name VARCHAR2,
        p_course VARCHAR2,
        p_marks NUMBER);

    PROCEDURE Retrieve_Student(p_id NUMBER);

    PROCEDURE Update_Student(
        p_id NUMBER,
        p_name VARCHAR2,
        p_course VARCHAR2,
        p_marks NUMBER);

    PROCEDURE Delete_Student(p_id NUMBER);

END Student_Pkg;
/

OUTPUT:
Package Created.

------------------------------------------------------------
STEP 3: CREATE PACKAGE BODY
------------------------------------------------------------

SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE BODY Student_Pkg AS

    PROCEDURE Insert_Student(
        p_id NUMBER,
        p_name VARCHAR2,
        p_course VARCHAR2,
        p_marks NUMBER) IS
    BEGIN
        INSERT INTO STUDENT
        VALUES (p_id, p_name, p_course, p_marks);
        DBMS_OUTPUT.PUT_LINE('Student Inserted Successfully');
    END;
```

```
    PROCEDURE Retrieve_Student(p_id NUMBER) IS
        v_name STUDENT.Student_name%TYPE;
        v_course STUDENT.Course%TYPE;
        v_marks STUDENT.Marks%TYPE;
    BEGIN
        SELECT Student_name, Course, Marks
        INTO v_name, v_course, v_marks
        FROM STUDENT
        WHERE Student_id = p_id;

        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
        DBMS_OUTPUT.PUT_LINE('Course: ' || v_course);
        DBMS_OUTPUT.PUT_LINE('Marks: ' || v_marks);
    END;

    PROCEDURE Update_Student(
        p_id NUMBER,
        p_name VARCHAR2,
        p_course VARCHAR2,
        p_marks NUMBER) IS
    BEGIN
        UPDATE STUDENT
        SET Student_name = p_name,
            Course = p_course,
            Marks = p_marks
        WHERE Student_id = p_id;

        DBMS_OUTPUT.PUT_LINE('Student Updated Successfully');
    END;

    PROCEDURE Delete_Student(p_id NUMBER) IS
    BEGIN
        DELETE FROM STUDENT
        WHERE Student_id = p_id;

        DBMS_OUTPUT.PUT_LINE('Student Deleted Successfully');
    END;

END Student_Pkg;
/

OUTPUT:
Package Body Created.

-------------------------------------------------------------
STEP 4: CALL PACKAGE PROCEDURES
-------------------------------------------------------------

EXEC Student_Pkg.Insert_Student(1,'Om','MCA',85);
EXEC Student_Pkg.Retrieve_Student(1);
EXEC Student_Pkg.Update_Student(1,'Omkar','MCA',90);
EXEC Student_Pkg.Delete_Student(1);

COMMIT;

-------------------------------------------------------------
EXPECTED OUTPUT:

Student Inserted Successfully
Name: Om
Course: MCA
Marks: 85
Student Updated Successfully
Student Deleted Successfully

-------------------------------------------------------------
VIVA POINTS:

1. Package Specification declares procedures/functions.
2. Package Body defines implementation.
3. Package improves modularity and code reusability.
4. EXEC is used to call stored procedures.
5. %TYPE inherits datatype from table column.
```

# PL/SQL Practical No. 14 - Exception Handling

```
QUESTION:

Write a program to illustrate:
1. User-defined exceptions
2. Built-in exceptions
3. Raise application error exceptions

------------------------------------------------------------
STEP 1: BUILT-IN EXCEPTION (ZERO_DIVIDE)
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 10;
    num2 NUMBER := 0;
    result NUMBER;
BEGIN
    result := num1 / num2;
    DBMS_OUTPUT.PUT_LINE('Result: ' || result);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Cannot divide by zero.');
END;
/

EXPECTED OUTPUT:
Error: Cannot divide by zero.

------------------------------------------------------------
STEP 2: USER-DEFINED EXCEPTION
------------------------------------------------------------

DECLARE
    age NUMBER := 15;
    invalid_age EXCEPTION;
BEGIN
    IF age < 18 THEN
        RAISE invalid_age;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Valid Age');

EXCEPTION
    WHEN invalid_age THEN
        DBMS_OUTPUT.PUT_LINE('Error: Age must be 18 or above.');
END;
/

EXPECTED OUTPUT:
Error: Age must be 18 or above.

------------------------------------------------------------
STEP 3: RAISE_APPLICATION_ERROR EXCEPTION
------------------------------------------------------------

DECLARE
    salary NUMBER := 2000;
BEGIN
    IF salary < 5000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary too low.');
    END IF;

    DBMS_OUTPUT.PUT_LINE('Salary is acceptable.');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
EXPECTED OUTPUT:
```

```
ORA-20001: Salary too low.
```

---

VIVA POINTS:

1. Built-in exceptions are predefined (ZERO_DIVIDE, NO_DATA_FOUND).
2. User-defined exceptions are declared using EXCEPTION keyword.
3. RAISE keyword is used to raise user-defined exception.
4. RAISE_APPLICATION_ERROR is used to create custom Oracle error.
5. SQLERRM returns error message text.

# PL/SQL Practical No. 15 - Reverse String Using PL/SQL Block

```
QUESTION:

Write a program to reverse a string using PL/SQL block.

------------------------------------------------------------
STEP 1: PL/SQL BLOCK
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    v_string VARCHAR2(50) := 'BHAVESH';
    v_reverse VARCHAR2(50) := '';
    i NUMBER;
BEGIN
    FOR i IN REVERSE 1..LENGTH(v_string) LOOP
        v_reverse := v_reverse || SUBSTR(v_string, i, 1);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Original String: ' || v_string);
    DBMS_OUTPUT.PUT_LINE('Reversed String: ' || v_reverse);
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:

Original String: BHAVESH
Reversed String: HSEVAHB

------------------------------------------------------------
VIVA POINTS:

1. LENGTH() returns length of string.
2. SUBSTR() extracts character from string.
3. REVERSE loop runs from last position to first.
4. || operator is used for string concatenation.
5. DBMS_OUTPUT.PUT_LINE displays output.
```

# PL/SQL Practical No. 16 - Trigger for Auditing Table Changes

```
QUESTION:

Create a trigger that records changes to an EMPLOYEES table
(INSERT, UPDATE, DELETE) into an EMPLOYEES_AUDIT table.
Include details like EMPLOYEE_ID, OPERATION_TYPE, TIMESTAMP.

-------------------------------------------------------------
STEP 1: CREATE EMPLOYEES TABLE
-------------------------------------------------------------

CREATE TABLE EMPLOYEES (
    Employee_ID NUMBER PRIMARY KEY,
    Employee_Name VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

-------------------------------------------------------------
STEP 2: CREATE AUDIT TABLE
-------------------------------------------------------------

CREATE TABLE EMPLOYEES_AUDIT (
    Employee_ID NUMBER,
    Operation_Type VARCHAR2(10),
    Operation_Time DATE
);

OUTPUT:
Table Created.

-------------------------------------------------------------
STEP 3: CREATE TRIGGER
-------------------------------------------------------------

CREATE OR REPLACE TRIGGER Employees_Audit_Trigger
AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO EMPLOYEES_AUDIT
        VALUES (:NEW.Employee_ID, 'INSERT', SYSDATE);

    ELSIF UPDATING THEN
        INSERT INTO EMPLOYEES_AUDIT
        VALUES (:NEW.Employee_ID, 'UPDATE', SYSDATE);

    ELSIF DELETING THEN
        INSERT INTO EMPLOYEES_AUDIT
        VALUES (:OLD.Employee_ID, 'DELETE', SYSDATE);
    END IF;
END;
/

OUTPUT:
Trigger Created.

-------------------------------------------------------------
STEP 4: TEST THE TRIGGER
-------------------------------------------------------------

INSERT INTO EMPLOYEES VALUES (1,'Om',30000);
UPDATE EMPLOYEES SET Salary = 35000 WHERE Employee_ID = 1;
DELETE FROM EMPLOYEES WHERE Employee_ID = 1;

COMMIT;

-------------------------------------------------------------
```

STEP 5: VERIFY AUDIT TABLE

```
SELECT * FROM EMPLOYEES_AUDIT;
```

EXPECTED OUTPUT:

| Employee_ID | Operation_Type | Operation_Time |
|---|---|---|
| 1 | INSERT | 22-FEB-26 |
| 1 | UPDATE | 22-FEB-26 |
| 1 | DELETE | 22-FEB-26 |

---

VIVA POINTS:

1. AFTER trigger executes after the DML operation.
2. INSERTING, UPDATING, DELETING are conditional predicates.
3. :NEW stores new values, :OLD stores old values.
4. Audit table keeps history of changes.
5. SYSDATE records current system date and time.

# PL/SQL Practical No. 17 - Employee Bonus Using Explicit Cursor

```
QUESTION:

Write a PL/SQL program using an explicit cursor to calculate and
display a 10% bonus for all employees whose salary is greater than 50,000.
Assume a table EMPLOYEES with columns EMPLOYEE_ID, NAME, and SALARY.

------------------------------------------------------------
STEP 1: CREATE EMPLOYEES TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEES (
    EMPLOYEE_ID NUMBER PRIMARY KEY,
    NAME VARCHAR2(50),
    SALARY NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEES VALUES (1,'Om',60000);
INSERT INTO EMPLOYEES VALUES (2,'Rahul',45000);
INSERT INTO EMPLOYEES VALUES (3,'Sneha',75000);
INSERT INTO EMPLOYEES VALUES (4,'Amit',52000);

COMMIT;

------------------------------------------------------------
STEP 3: PL/SQL PROGRAM USING EXPLICIT CURSOR
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    CURSOR emp_cursor IS
        SELECT EMPLOYEE_ID, NAME, SALARY
        FROM EMPLOYEES
        WHERE SALARY > 50000;

    v_id EMPLOYEES.EMPLOYEE_ID%TYPE;
    v_name EMPLOYEES.NAME%TYPE;
    v_salary EMPLOYEES.SALARY%TYPE;
    v_bonus NUMBER;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        v_bonus := v_salary * 0.10;

        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
        DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
        DBMS_OUTPUT.PUT_LINE('Bonus (10%): ' || v_bonus);
        DBMS_OUTPUT.PUT_LINE('----------------------------');
    END LOOP;

    CLOSE emp_cursor;
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:
Employee ID: 1
```

```
Name: Om
Salary: 60000
Bonus (10%): 6000

Employee ID: 3
Name: Sneha
Salary: 75000
Bonus (10%): 7500

Employee ID: 4
Name: Amit
Salary: 52000
Bonus (10%): 5200

------------------------------------------------------------
VIVA POINTS:

1. Explicit cursor is declared using CURSOR keyword.
2. OPEN, FETCH, and CLOSE are required for explicit cursor.
3. %NOTFOUND checks if no more rows are available.
4. Bonus is calculated using salary * 0.10.
5. DBMS_OUTPUT.PUT_LINE displays output.
```

# SQL Practical No. 18 - Implementation of Aggregate Functions

```
QUESTION:

Write a SQL program to implement Aggregate Functions.

------------------------------------------------------------
STEP 1: CREATE EMPLOYEE TABLE
------------------------------------------------------------

CREATE TABLE EMPLOYEE (
    Emp_id NUMBER PRIMARY KEY,
    Emp_name VARCHAR2(50),
    Department VARCHAR2(50),
    Salary NUMBER
);

OUTPUT:
Table Created.

------------------------------------------------------------
STEP 2: INSERT SAMPLE DATA
------------------------------------------------------------

INSERT INTO EMPLOYEE VALUES (1,'Om','IT',50000);
INSERT INTO EMPLOYEE VALUES (2,'Rahul','HR',30000);
INSERT INTO EMPLOYEE VALUES (3,'Sneha','IT',60000);
INSERT INTO EMPLOYEE VALUES (4,'Amit','Finance',40000);
INSERT INTO EMPLOYEE VALUES (5,'Neha','HR',35000);

COMMIT;

------------------------------------------------------------
STEP 3: APPLY AGGREGATE FUNCTIONS
------------------------------------------------------------

1) COUNT() – Count total employees

SELECT COUNT(*) AS Total_Employees FROM EMPLOYEE;

Expected Output:
Total_Employees
5

------------------------------------------------------------

2) SUM() – Total salary of employees

SELECT SUM(Salary) AS Total_Salary FROM EMPLOYEE;

Expected Output:
Total_Salary
215000

------------------------------------------------------------

3) AVG() – Average salary

SELECT AVG(Salary) AS Average_Salary FROM EMPLOYEE;

Expected Output:
Average_Salary
43000

------------------------------------------------------------

4) MAX() – Highest salary

SELECT MAX(Salary) AS Highest_Salary FROM EMPLOYEE;

Expected Output:
```

```
Highest_Salary
60000
```

---

5) MIN() – Lowest salary

```
SELECT MIN(Salary) AS Lowest_Salary FROM EMPLOYEE;
```

Expected Output:

```
Lowest_Salary
30000
```

---

6) Aggregate with GROUP BY – Department-wise total salary

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM EMPLOYEE
GROUP BY Department;
```

Expected Output:

```
Department | Total_Salary
IT         | 110000
HR         | 65000
Finance    | 40000
```

---

VIVA POINTS:

1. Aggregate functions operate on multiple rows and return a single value.
2. Common aggregate functions: COUNT, SUM, AVG, MAX, MIN.
3. GROUP BY is used with aggregate functions.
4. NULL values are ignored in aggregate functions (except COUNT(*)).
5. Aggregate functions are used in SELECT statement.

# PL/SQL Practical No. 19 - Finding Even Numbers

```
QUESTION:

Write PL/SQL code for finding even numbers.

------------------------------------------------------------
STEP 1: PL/SQL BLOCK TO DISPLAY EVEN NUMBERS FROM 1 TO 20
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    i NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Even Numbers from 1 to 20 are:');

    FOR i IN 1..20 LOOP
        IF MOD(i, 2) = 0 THEN
            DBMS_OUTPUT.PUT_LINE(i);
        END IF;
    END LOOP;
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:

Even Numbers from 1 to 20 are:
2
4
6
8
10
12
14
16
18
20

------------------------------------------------------------
ALTERNATIVE METHOD (Using WHILE Loop)
------------------------------------------------------------

DECLARE
    i NUMBER := 2;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Even Numbers from 1 to 20 are:');

    WHILE i <= 20 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 2;
    END LOOP;
END;
/

------------------------------------------------------------
VIVA POINTS:

1. MOD(number, 2) = 0 checks even number.
2. FOR loop runs from starting to ending value.
3. WHILE loop runs until condition becomes false.
4. DBMS_OUTPUT.PUT_LINE displays output.
5. SET SERVEROUTPUT ON enables output display.
```

# PL/SQL Practical No. 20 - Largest of Three Numbers

```
QUESTION:

Write PL/SQL code to find the largest of three numbers.

------------------------------------------------------------
METHOD 1: USING IF-ELSIF-ELSE
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 10;
    num2 NUMBER := 25;
    num3 NUMBER := 15;
    largest NUMBER;
BEGIN
    IF num1 >= num2 AND num1 >= num3 THEN
        largest := num1;
    ELSIF num2 >= num1 AND num2 >= num3 THEN
        largest := num2;
    ELSE
        largest := num3;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Largest Number is: ' || largest);
END;
/

------------------------------------------------------------
EXPECTED OUTPUT:

Largest Number is: 25

------------------------------------------------------------
METHOD 2: TAKING USER INPUT
------------------------------------------------------------

DECLARE
    num1 NUMBER;
    num2 NUMBER;
    num3 NUMBER;
    largest NUMBER;
BEGIN
    num1 := &Enter_First_Number;
    num2 := &Enter_Second_Number;
    num3 := &Enter_Third_Number;

    IF num1 >= num2 AND num1 >= num3 THEN
        largest := num1;
    ELSIF num2 >= num1 AND num2 >= num3 THEN
        largest := num2;
    ELSE
        largest := num3;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Largest Number is: ' || largest);
END;
/

------------------------------------------------------------
VIVA POINTS:

1. IF-ELSIF-ELSE is used for decision making.
2. AND operator checks multiple conditions.
3. >= ensures correct comparison even if numbers are equal.
4. & is used for user input in SQL*Plus.
5. DBMS_OUTPUT.PUT_LINE displays output.
```

# PL/SQL Practical No. 16 - Trigger for Auditing Table Changes

```
QUESTION:
Create a trigger that records changes (INSERT, UPDATE, DELETE) on EMPLOYEES table
into EMPLOYEES_AUDIT table.

------------------------------------------------------------
TABLE CREATION:
------------------------------------------------------------

CREATE TABLE EMPLOYEES (
    EMPLOYEE_ID NUMBER PRIMARY KEY,
    ENAME VARCHAR2(50),
    SALARY NUMBER
);

CREATE TABLE EMPLOYEES_AUDIT (
    EMPLOYEE_ID NUMBER,
    OPERATION_TYPE VARCHAR2(10),
    ACTION_DATE TIMESTAMP
);

------------------------------------------------------------
TRIGGER CREATION:
------------------------------------------------------------

CREATE OR REPLACE TRIGGER trg_employees_audit
AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO EMPLOYEES_AUDIT (EMPLOYEE_ID, OPERATION_TYPE, ACTION_DATE)
        VALUES (:NEW.EMPLOYEE_ID, 'INSERT', SYSTIMESTAMP);

    ELSIF UPDATING THEN
        INSERT INTO EMPLOYEES_AUDIT (EMPLOYEE_ID, OPERATION_TYPE, ACTION_DATE)
        VALUES (:NEW.EMPLOYEE_ID, 'UPDATE', SYSTIMESTAMP);

    ELSIF DELETING THEN
        INSERT INTO EMPLOYEES_AUDIT (EMPLOYEE_ID, OPERATION_TYPE, ACTION_DATE)
        VALUES (:OLD.EMPLOYEE_ID, 'DELETE', SYSTIMESTAMP);
    END IF;
END;
/

------------------------------------------------------------
TESTING:
------------------------------------------------------------

-- Insert
INSERT INTO EMPLOYEES VALUES (1, 'John', 50000);

-- Update
UPDATE EMPLOYEES SET SALARY = 55000 WHERE EMPLOYEE_ID = 1;

-- Delete
DELETE FROM EMPLOYEES WHERE EMPLOYEE_ID = 1;

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

SELECT * FROM EMPLOYEES_AUDIT;

EMPLOYEE_ID   OPERATION_TYPE   ACTION_DATE
-----------------------------------------
1             INSERT           (timestamp)
1             UPDATE           (timestamp)
1             DELETE           (timestamp)
```

# PL/SQL Practical No. 21 - Reverse Text and Check Palindrome

```
QUESTION:
Write PL/SQL code to accept text, reverse it, and check whether it is a palindrome or not.

------------------------------------------------------------
CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    str VARCHAR2(50);
    rev VARCHAR2(50) := '';
    i NUMBER;
BEGIN
    -- Accept input
    str := '&Enter_String';

    -- Reverse string
    FOR i IN REVERSE 1..LENGTH(str) LOOP
        rev := rev || SUBSTR(str, i, 1);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Original String: ' || str);
    DBMS_OUTPUT.PUT_LINE('Reversed String: ' || rev);

    -- Check Palindrome
    IF str = rev THEN
        DBMS_OUTPUT.PUT_LINE('It is a Palindrome');
    ELSE
        DBMS_OUTPUT.PUT_LINE('It is NOT a Palindrome');
    END IF;
END;
/

------------------------------------------------------------
OUTPUT (Example):
------------------------------------------------------------

Enter value for Enter_String: madam

Original String: madam
Reversed String: madam
It is a Palindrome
```

# PL/SQL Practical No. 22 - Insert Values into Table

```
QUESTION:
Write PL/SQL code to insert values into created tables.

------------------------------------------------------------
TABLE CREATION:
------------------------------------------------------------

CREATE TABLE STUDENT (
    ID NUMBER PRIMARY KEY,
    NAME VARCHAR2(50),
    COURSE VARCHAR2(30)
);

------------------------------------------------------------
PL/SQL CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

BEGIN
    INSERT INTO STUDENT VALUES (1, 'Rahul', 'MCA');
    INSERT INTO STUDENT VALUES (2, 'Anita', 'BCA');
    INSERT INTO STUDENT VALUES (3, 'Amit', 'MBA');

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Records Inserted Successfully');
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Records Inserted Successfully

------------------------------------------------------------
VERIFY DATA:
------------------------------------------------------------

SELECT * FROM STUDENT;

ID   NAME    COURSE
----------------------
1    Rahul   MCA
2    Anita   BCA
3    Amit    MBA
```

# PL/SQL Practical No. 23 - Update Using Implicit Cursor

```
QUESTION:
Write PL/SQL code to UPDATE values in created tables using Implicit Cursors.

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

STUDENT(ID, NAME, COURSE)

------------------------------------------------------------
PL/SQL CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

BEGIN
    UPDATE STUDENT
    SET COURSE = 'MCA'
    WHERE ID = 2;

    -- Using implicit cursor attribute
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' Record(s) Updated Successfully');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No Records Updated');
    END IF;

    COMMIT;
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

1 Record(s) Updated Successfully

------------------------------------------------------------
VERIFY DATA:
------------------------------------------------------------

SELECT * FROM STUDENT;

ID   NAME    COURSE
----------------------
1    Rahul   MCA
2    Anita   MCA
3    Amit    MBA
```

# PL/SQL Practical No. 24 - Display Employee Details using Explicit Cursor

```
QUESTION:
Write PL/SQL code to display Employee details using Explicit Cursor.

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

EMPLOYEES(EMPLOYEE_ID, ENAME, SALARY)

------------------------------------------------------------
PL/SQL CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    CURSOR emp_cursor IS
        SELECT EMPLOYEE_ID, ENAME, SALARY FROM EMPLOYEES;

    v_id EMPLOYEES.EMPLOYEE_ID%TYPE;
    v_name EMPLOYEES.ENAME%TYPE;
    v_salary EMPLOYEES.SALARY%TYPE;

BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('ID: ' || v_id ||
                             ' Name: ' || v_name ||
                             ' Salary: ' || v_salary);
    END LOOP;

    CLOSE emp_cursor;
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

ID: 1 Name: John Salary: 50000
ID: 2 Name: Alice Salary: 60000
```

# PL/SQL Practical No. 25 - Display Employee Name and Salary using Cursor

```
QUESTION:
Write PL/SQL code using Cursor to display employee names and salary.

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

EMPLOYEES(EMPLOYEE_ID, ENAME, SALARY)

------------------------------------------------------------
PL/SQL CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    CURSOR emp_cursor IS
        SELECT ENAME, SALARY FROM EMPLOYEES;

    v_name EMPLOYEES.ENAME%TYPE;
    v_salary EMPLOYEES.SALARY%TYPE;

BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name ||
                             ' Salary: ' || v_salary);
    END LOOP;

    CLOSE emp_cursor;
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Name: John Salary: 50000
Name: Alice Salary: 60000
```

# PL/SQL Practical No. 26 - Using Two Cursors

```
QUESTION:
Write PL/SQL Program using two cursors at a time.

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

EMPLOYEES(EMPLOYEE_ID, ENAME, SALARY)

------------------------------------------------------------
PL/SQL CODE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

DECLARE
    -- First Cursor
    CURSOR c1 IS
        SELECT ENAME FROM EMPLOYEES;

    -- Second Cursor
    CURSOR c2 IS
        SELECT SALARY FROM EMPLOYEES;

    v_name EMPLOYEES.ENAME%TYPE;
    v_salary EMPLOYEES.SALARY%TYPE;

BEGIN
    OPEN c1;
    OPEN c2;

    LOOP
        FETCH c1 INTO v_name;
        FETCH c2 INTO v_salary;

        EXIT WHEN c1%NOTFOUND OR c2%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name ||
                             ' Salary: ' || v_salary);
    END LOOP;

    CLOSE c1;
    CLOSE c2;
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Name: John Salary: 50000
Name: Alice Salary: 60000
```

# PL/SQL Practical No. 27 - Procedure to Reverse a Number

```
QUESTION:
Write PL/SQL code using Procedure to reverse a number.

------------------------------------------------------------
PROCEDURE CREATION:
------------------------------------------------------------

CREATE OR REPLACE PROCEDURE reverse_num (n IN NUMBER) IS
    num NUMBER;
    rev NUMBER := 0;
    rem NUMBER;
BEGIN
    num := n;

    WHILE num > 0 LOOP
        rem := MOD(num, 10);
        rev := rev * 10 + rem;
        num := TRUNC(num / 10);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Reversed Number: ' || rev);
END;
/

------------------------------------------------------------
CALL PROCEDURE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

BEGIN
    reverse_num(1234);
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Reversed Number: 4321
```

# PL/SQL Practical No. 28 - Procedure to Find Factorial

```
QUESTION:
Write PL/SQL code using Procedure to find factorial of a given number.

------------------------------------------------------------
PROCEDURE CREATION:
------------------------------------------------------------

CREATE OR REPLACE PROCEDURE factorial_proc (n IN NUMBER) IS
    fact NUMBER := 1;
    i NUMBER;
BEGIN
    FOR i IN 1..n LOOP
        fact := fact * i;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Factorial of ' || n || ' is ' || fact);
END;
/

------------------------------------------------------------
CALL PROCEDURE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

BEGIN
    factorial_proc(5);
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Factorial of 5 is 120
```

# PL/SQL Practical No. 29 - Procedure to Retrieve Employee Salary

```
QUESTION:
Write a procedure to retrieve the salary of a particular employee.

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

EMPLOYEES(EMPLOYEE_ID, ENAME, SALARY)

------------------------------------------------------------
PROCEDURE CREATION:
------------------------------------------------------------

CREATE OR REPLACE PROCEDURE get_salary (
    emp_id IN NUMBER
) IS
    v_salary EMPLOYEES.SALARY%TYPE;
BEGIN
    SELECT SALARY INTO v_salary
    FROM EMPLOYEES
    WHERE EMPLOYEE_ID = emp_id;

    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/

------------------------------------------------------------
CALL PROCEDURE:
------------------------------------------------------------

SET SERVEROUTPUT ON;

BEGIN
    get_salary(1);
END;
/

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

Salary: 50000
(or)
Employee not found
```

# PL/SQL Practical No. 30 - Trigger to Prevent Duplicate EMPNO

```
QUESTION:
Write PL/SQL code using Trigger to not accept existing EMPNO (unique number).

------------------------------------------------------------
TABLE (Assume Already Created):
------------------------------------------------------------

EMPLOYEES(EMPLOYEE_ID, ENAME, SALARY)

------------------------------------------------------------
TRIGGER CREATION:
------------------------------------------------------------

CREATE OR REPLACE TRIGGER trg_check_empno
BEFORE INSERT ON EMPLOYEES
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM EMPLOYEES
    WHERE EMPLOYEE_ID = :NEW.EMPLOYEE_ID;

    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'EMPLOYEE_ID already exists');
    END IF;
END;
/

------------------------------------------------------------
TESTING:
------------------------------------------------------------

-- First Insert (Works)
INSERT INTO EMPLOYEES VALUES (1, 'John', 50000);

-- Duplicate Insert (Fails)
INSERT INTO EMPLOYEES VALUES (1, 'Alice', 60000);

------------------------------------------------------------
OUTPUT:
------------------------------------------------------------

ORA-20001: EMPLOYEE_ID already exists
```